



1. Problem Definition and Classification

The Capacitated Vehicle Routing Problem (CVRP) is the problem of determining optimal routes for vehicles delivering goods to a set of customers.

Things to be aware of would be:

1. Each customer is visited once
2. Vehicle capacity is respected
3. Total cost to drive is minimized

Classification: NP-Hard

Justification: VRP is a generalization of the Traveling salesmen problem, which is a NP-Hard problem. Since it adds additional constraints, it must be at least as hard as TSP.

Real-World Applications:

1. Logistics (Amazon, UPS routing)
2. Ride-sharing (Uber drivers)
3. Dump truck routing
4. Ambulance dispatch routing



2. Algorithm Selection

Algorithm 1 (benchmark): **Greedy Nearest Neighbor**

Concept: Start at the warehouse and always go to the closest unvisited customer. Upon choosing the nearest customer, it will evaluate if it is within the capacity of the vehicle; it will do as such until it is no longer within the capacity. When it has reached the bound of its capacity, it will return to the depot and begin the algorithm again skipping over already visited customers until all customers have been visited, ending the current algorithm and returning to the warehouse.

Time Complexity: $\theta(n^2)$

Strengths:

1. Fairly fast
2. Easy to implement
3. Guaranteed to produce a feasible solution

Weaknesses:

1. Get stuck on bad routes
2. As the algorithm continues, it will create less optimal paths
3. We include the algorithm as a baseline to compare all other algorithms against it.

Algorithm 2: **Clarke-Wright Saving Algorithm**

Concept: Start with separate routes and then merge them until they reach the capacity of the vehicle. Upon doing so, it will list the different routes in order based upon travel distance saved.

Time complexity: $\theta^2 \log(n)$

Strengths:

1. Fast route generation
2. Vary efficient (Only multiplying by $\log n$)

Weaknesses:

1. Once two routes are merged, they cannot be undone, meaning any bad merges made early on will carry through to the final solution.



2. As the number of customers increases, solution quality decreases since the algorithm has no way to go back and reconsider earlier merges it has already committed to.

We include this algorithm because it is VRP-specific and very industry-related.

Algorithm 3: **Local Search**

Concept: Start with a solution and then improve my swapping edges through targeted modifications

Time Complexity: $\theta(n^2)$

Strengths:

1. Easy to implement with existing solutions
2. Fast iterations on paths

Weaknesses:

1. Can get stuck on a local optimal path rather than working towards the global optimal path
2. Dependent on starting position for routes, and rerunning new starting positions increases runtime proportionally

We include the algorithms because it represents improvement-based solutions

Algorithm 4: **Ant Colony Optimization**

Concept: Simulates ants laying pheromones to find shortest routes, reinforcing commonly used routes by other ants

Time complexity: $\theta(\text{iterations} \times \text{ants} \times n^2)$

Strengths:

1. Reinforces high quality routes
2. Good for complex VRP

Weaknesses

1. Can reinforce bad routes when pheromone concentration is set early on
2. Slower on smaller problems where constant integration offers little advantages

We include this algorithm because it represents nature inspired solutions and shows trade offs



3. Research Questions

1. How does runtime scale as input size (n) increases

As the number of customers increases, each algorithm's runtime grows at different rates:

Greedy Nearest Neighbor:

- For each step, you scan remaining nodes: n comparisons each
- Growth behavior: Quadratic
 - This means that the algorithm is fast for small/medium n but becomes a lot slower as n gets large.

Clarke-Wright Savings:

- Computers savings for all pairs: n^2 , Then it sorts the savings: $\log n$ factor
- Growth behavior: Very slightly worse than quadratic
 - This means its slightly slower than greedy but still efficient
 - It scales up well to a large n

Local Search:

- Each iteration checks pairs of edges: n^2
- Total runtime depends on number of iterations
- Growth behavior:
 - Roughly $\theta(k * n^2)$, where k is the iterations
 - This means that it can become very slow if many iterations are needed.

2. Trade-off between solution quality and execution time

Greedy Nearest Neighbor:

- Fast compared to other algorithms
- Does not have very good quality solutions

Clarke-Wright:

- Moderate runtime
- Better solutions than Greedy



Local Search:

- Slower than greedy
- Improves solutions significantly

ACO:

- Slowest of our algorithms
- Best solutions (on average) due to the complexity

3. At what input size does ACO outperform others:

ACO begins to outperform greedy and Clarke–Wright in terms of solution quality at moderate input sizes (approximately $n \geq 50-100$), where the search space becomes large enough for better solution exploration to be beneficial.

However, ACO does not outperform these algorithms in runtime, as it has significantly higher computational cost due to repeated simulations and parameter tuning.

Also, things we need to specify:

- Input sizes are simply the n amount of customers
- Parameters will vary however things to be aware of would be:
 - o Number of Vehicles
 - o Vehicle Capacity
 - o ACO parameters would be number of pheromones, perhaps time it takes for them to go away
- Metrics we need to measure:
 - o Runtime
 - o Memory Usage
 - o Total route distance
- Trials:
 - o 5-10 runs per configuration



Minimum and Maximum of our Project:

At minimum, we will run all four algorithms on a synthetic dataset of $n = 50$ customers with default parameters and compare total route distance and runtime for each algorithm. This guarantees a working implementation regardless of time constraints.

At our most ambitious, we will scale all four algorithms up to $n = 500$, perform parameter sweeps for ACO's evaporation rate and number of ants, and compare our results against published near-optimal solutions from the Solomon benchmark instances.



4. Dataset Plan

Synthetic Dataset:

- Generating a synthetic dataset with a grid size of $[0, x]$ by $[y, 0]$ with x and y being scalable for generation. Upon a grid's generation multiple customer locations (C) will be generated within the grid's bounds (ex. $[1, 10]$, $[32, 4]$, ...) up to a specified amount before generation. The depot location will be at $[x/2, y/2]$ placing it at the center of the grid. The vehicle capacity being initially set at $Q=25$ but scalable up to the upper bound of customer generations (C).

5. Risk Assessment

The potential risk we estimate to run into is implementation of the Ant Colony Optimization. Everything within the project is expected to be within the scope of our current skills and applicable knowledge, leading to the only concern being time management. In preparation for time being our biggest concern, we are delegating the creation of each algorithm to everyone, one per individual. We have decided to sit down and collaborate on creating the data set early as we see it as a necessity for testing algorithms as we are creating them.

Parsing and formatting our dataset into a consistent structure usable by all four algorithms will require early team coordination to agree on a shared input format before implementation begins.

ACO is sensitive to parameters such as evaporation rate and number of ants, so finding a working configuration will require trial and error upfront before we can run reliable experiments. Overall, the roadblocks concerning the project seem simple to overcome, and hiccups as such are not foreseen but will be preemptively planned for.